

Lab 5: A Defective Filament

Variable-Coefficient PDEs: FDM vs. PINNs

Spring 2026

Learning Objectives

By completing this lab, you will:

- Use Object-Oriented Programming to supply physics parameters to a solver.
- Understand the charge continuity equation and Ohm's law.
- Implement a staggered-grid Finite Difference Method (FDM) for a variable-coefficient PDE.
- Use PyTorch automatic differentiation to solve the same problem via a PINN.
- Discover the physical mechanism behind why light bulbs burn out at specific spots.

Part 1: Steady Current in a Non-Uniform Wire

1.1 Deriving the Governing Equation

In class, we used the Drude model to define conductivity σ . Now, consider a 1D tungsten filament of length L . We apply a voltage across it, creating an electric potential $V(x)$. The electric field is the negative slope of the potential:

$$E(x) = -\frac{dV}{dx}$$

By Ohm's law, the current density $J(x)$ is:

$$J(x) = \sigma(x)E(x) = -\sigma(x)\frac{dV}{dx}$$

For a steady-state DC circuit, charge cannot pile up anywhere. The charge continuity equation requires that the current is constant across the entire wire. If the current is constant, its spatial derivative is zero:

$$\frac{dJ}{dx} = 0 \implies \frac{d}{dx} \left(\sigma(x) \frac{dV}{dx} \right) = 0$$

This is a **variable-coefficient Poisson equation**. If σ were constant, it would pull out of the derivative, leaving Laplace's equation $\frac{d^2V}{dx^2} = 0$, which just gives a linear voltage drop. But our filament has a defect!

1.2 The Defect

You will use your `Filament` class from the in-class exercise. The conductivity $\sigma(x)$ dips by 50% in the middle of the wire due to a manufacturing impurity.

The Boundary Conditions

We are hooking the filament up to a standard US outlet voltage (we'll treat it as DC for this snapshot):

- $V(0) = 120$ Volts
- $V(L) = 0$ Volts

Part 2: Staggered Finite Differences

Solving $\frac{d}{dx} \left(\sigma(x) \frac{dV}{dx} \right) = 0$ using traditional grids is tricky. If we just use standard centered differences, taking the derivative of a product gets very messy.

Instead, we use a **staggered grid**. Think of the wire as a series of nodes $(i-1, i, i+1)$. We calculate the Voltage V directly on these nodes. But we calculate the Conductivity σ exactly halfway between the nodes $(i-1/2, i+1/2)$.

2.1 The Staggered Math

Step 1: Calculate the first derivative $\frac{dV}{dx}$ halfway between node i and $i + 1$.

$$\left(\frac{dV}{dx}\right)_{i+1/2} \approx \frac{V_{i+1} - V_i}{\Delta x}$$

Step 2: Multiply by the conductivity at that exact same half-step to get the current density J .

$$J_{i+1/2} = -\sigma_{i+1/2} \left(\frac{V_{i+1} - V_i}{\Delta x}\right)$$

Step 3: Finally, take the derivative of J across node i .

$$\frac{dJ}{dx} \approx \frac{J_{i+1/2} - J_{i-1/2}}{\Delta x} = 0$$

Substitute Step 2 into Step 3, drop the negative signs, multiply out the Δx , and we get our final matrix equation:

$$\sigma_{i-1/2}V_{i-1} - (\sigma_{i+1/2} + \sigma_{i-1/2})V_i + \sigma_{i+1/2}V_{i+1} = 0$$

2.2 Exercise 1: Building the FDM Matrix

This equation forms a tridiagonal system $AV = \mathbf{b}$. Write a function `solve_fdm(filament, N)` that builds this $N \times N$ matrix and solves for $V(x)$ using `np.linalg.solve`.

```

1  def solve_fdm(filament, N=100):
2      x = np.linspace(0, filament.L, N)
3      dx = x[1] - x[0]
4
5      # Evaluate sigma at the N-1 half steps
6      # sig_half[i] represents the conductivity at i+1/2
7      x_half = x[:-1] + dx/2
8      sig_half = np.array([filament.sigma(xi) for xi in x_half])
9
10     A = np.zeros((N, N))
11     b = np.zeros(N)
12
13     # Boundary Conditions
14     A[0, 0] = 1.0; b[0] = 120.0
15     A[-1, -1] = 1.0; b[-1] = 0.0
16
17     # TODO: Loop from i=1 to N-2.
18     # Use sig_half[i-1] for sigma_{i-1/2}
19     # Use sig_half[i] for sigma_{i+1/2}
20
21     V = np.linalg.solve(A, b)
22     return x, V

```

Listing 1: Skeleton for FDM Solver

Part 3: PINNs to the Rescue

Building that staggered matrix was annoying. If we move to 2D or 3D, keeping track of half-steps becomes a nightmare.

This is where Physics-Informed Neural Networks shine. PyTorch's automatic differentiation doesn't care about grids. It evaluates the function continuously and natively applies the product rule for us.

3.1 Exercise 2: PINN Residuals

We want to minimize the PDE residual: $\text{res} = \frac{dJ}{dx} = \frac{d}{dx} \left(\sigma(x) \frac{dV}{dx} \right)$.

The trick here is that PyTorch's `autograd.grad` simply takes a tensor and differentiates it with respect to its inputs. You just need to call it twice, injecting the conductivity $\sigma(x)$ in the middle! Complete the PyTorch residual function below.

```

1  def compute_pde_residual(model, x, filament):
2      # Tell PyTorch to track operations on x
3      x.requires_grad_(True)
4
5      # 1. Get Voltage from neural net
6      V = model(x)
7
8      # 2. First derivative: E-field = -dV/dx
9      # We use create_graph=True so we can differentiate again!
10     V_x = torch.autograd.grad(
11         V, x, grad_outputs=torch.ones_like(V),
12         create_graph=True
13     )[0]
14
15     # 3. Get sigma(x) as a tensor
16     sig = filament.sigma(x)
17
18     # 4. Current Density (Ohm's Law)
19     J = sig * V_x
20
21     # 5. Calculate divergence of J (dJ/dx)
22     # Notice we differentiate J this time, not V_x!
23     # TODO: Write the autograd function for J_x
24     # J_x = torch.autograd.grad( ... )[0]
25
26     return J_x

```

Listing 2: PINN Residual for Variable Coefficients

3.2 Exercise 3: The Training Loop

Since this is a 1D steady-state problem (no time dependence), creating collocation points is easier than in Lab 3. We just need random points along the x -axis.

Fill in the missing pieces of the training loop below. Pay close attention to your tensor shapes! Boundary points must be 1×1 tensors.

```

1  # --- PINN Training Setup ---
2  pinn_model = PDE_Net()
3  optimizer = torch.optim.Adam(pinn_model.parameters(), lr=0.01)
4  bulb = Filament(Tungsten())
5  epochs = 3000
6
7  # Generate boundary points (Tensors require shape N x 1)
8  x_left = torch.tensor([[0.0]])
9  x_right = torch.tensor([[bulb.L]])

```

```

10
11 loss_history = []
12
13 for epoch in range(epochs):
14     optimizer.zero_grad()
15
16     # 1. Domain Loss (The Physics)
17     # TODO: Generate 200 random collocation points between 0 and bulb.L
18     # x_domain = ...
19
20     # TODO: Compute the PDE residual using the function you wrote above
21     # residual = ...
22
23     # TODO: Calculate the Mean Squared Error of the residual
24     # loss_pde = ...
25
26
27     # 2. Boundary Loss (The Circuit)
28     # TODO: Predict voltage at x_left. Calculate MSE against target (120V)
29     # loss_bc_left = ...
30
31     # TODO: Predict voltage at x_right. Calculate MSE against target (0V)
32     # loss_bc_right = ...
33
34
35     # 3. Optimize
36     # TODO: Combine losses, call backward(), and step the optimizer
37     # loss_total = loss_pde + loss_bc_left + loss_bc_right
38     # ...
39     # ...
40
41     loss_history.append(loss_total.item())
42
43     if epoch % 500 == 0:
44         print(f"Epoch {epoch} | Loss: {loss_total.item():.4f}")

```

Listing 3: Scaffolded PINN Training Loop

Part 4: The Physics Payoff

Once both models are working, calculate the Electric Field $E(x) = -\frac{dV}{dx}$ and the Joule Heating density $S(x) = \sigma(x)E(x)^2$.

Questions

1. **Plot the Electric Field $E(x)$ across the filament.** What happens inside the defect region? Why must the physics behave this way to maintain a steady current?
2. **Plot the Joule Heating $S(x)$.** Where is the filament the hottest? Based on this, explain why old incandescent light bulbs always burn out with a sudden "pop" at a single tiny spot rather than melting uniformly.
3. **Compare the methods.** If the defect width w was much smaller than your FDM grid spacing Δx , what would happen to the FDM solution? Does the PINN suffer from this same grid limitation?